

PRG 222-----{

Step	Action	Command / Details
1	Install AWS CLI	bash\ncurl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"\nunzip awscliv2.zip\nsudo ./aws/install\n
2	Create S3 Bucket (Console)	AWS Console → S3 → Create bucket → General Purpose → Enter unique bucket name → Create
3	Upload Files	Open Bucket → Upload → Select files/folders → Upload
4	Create Access Key	IAM → Users → Security Credentials → Access Keys → Create Access Key → Select CLI → Save Access Key ID & Secret Access Key
5	Configure AWS CLI	bash\naws configure\n\nEnter:• AWS Access Key ID• AWS Secret Access Key• Default region: ap-south-1• Default output: text
6	Create Bucket using CLI	bash\naws s3api create-bucket --bucket <bucket-name> --region ap-south-1 --create-bucket-configuration LocationConstraint=ap-south-1\n
7	List Buckets	bash\naws s3api list-buckets\n
8	View Bucket Contents	bash\naws s3 ls s3://<bucket-name>\n
9	Copy File Between Buckets	bash\naws s3 cp s3://source-bucket/file.txt s3://destination-bucket/file.txt\n
10	Create CloudFront Distribution	CloudFront → Create Distribution → Select S3 Bucket → Create OAC → Create Distribution
11	Edit Bucket Policy	S3 → Bucket → Permissions → Bucket Policy → Edit → Paste the JSON below and replace <bucket-name>, <account-id>, and <distribution-id>.
12	Access File	https://<CloudFront-Domain-Name>/<file-name>

"Version": "2012-10-17",

"Statement": [

{

"Sid": "AllowCloudFrontAccessOnly",

"Effect": "Allow",

"Principal": {

"Service": "cloudfront.amazonaws.com"

},

"Action": "s3:GetObject",

"Resource": "arn:aws:s3:::<bucket-name>/*",

"Condition": {

"StringEquals": {

"AWS:SourceArn": "arn:aws:cloudfront::<account-id>:distribution/<distribution-id>"

}

}

}

]

}

Step	Action	Procedure / Commands	Purpose / Expected Output
1	Create VPC	AWS Console → VPC → Create VPC → VPC Only → Enter VPC Name and IPv4 CIDR (e.g., 10.0.0.0/16) → Create	Creates a private virtual network to host AWS resources.
2	Create Internet Gateway (IGW)	Internet Gateways → Create Internet Gateway → Give Name → Create → Attach to VPC	Provides Internet connectivity to the VPC.
3	Create Two Subnets	Subnets → Create Subnet → Select VPC → Enter Subnet Name → Choose Availability Zone → Enter IPv4 CIDR (e.g., 10.0.1.0/24) → Create. Repeat for second subnet.	Creates two different subnets for two EC2 instances.
4	Create Route Table	Route Tables → Create Route Table → Enter Name → Select VPC → Create	Route table controls network traffic in the VPC.
5	Associate Route Table	Route Table → Subnet Associations → Edit → Select both subnets → Save	Associates both subnets with the route table.
6	Connect Route Table to Internet	Route Table → Routes → Edit Routes → Add Route → Destination: 0.0.0.0/0 → Target: Internet Gateway → Save	Enables Internet access for instances inside the subnets.
7	Launch EC2 Instances (Create 2)	EC2 → Launch Instance → Ubuntu AMI → t2.micro → Create Key Pair → Network: Select VPC → Select first subnet → Enable Auto Assign Public IP → Security Group: Allow HTTP (80) from Anywhere → Launch. Repeat for second subnet.	Creates two web servers in different subnets.
8	Install Apache (User Data Script)	Paste the following in Advanced Details → User Data before launching the instance: <pre> bash\n#!/bin/bash\nsudo apt update -y\nsudo apt install -y apache2\nsudo systemctl start apache2\nsudo systemctl enable apache2\nnecho " <h1>Server Details</h1><p>Hostname: \$(hostname)</p><p>IP Address: \$(hostname -I cut -d' ' -f1)</p>" > /var/www/html/index.html\n\nsudo systemctl restart apache2\n </pre>	Automatically installs Apache, starts it, and creates a webpage showing the server hostname and IP address.

9	Verify Apache	EC2 Dashboard → Select Instance → Copy Public IPv4 Address → Open browser → <code>http://<Public-IP></code>	If Apache is working, the browser displays Server Details, Hostname, and IP Address . Repeat for both EC2 instances.
10	Create Target Group	EC2 Dashboard → Load Balancing → Target Groups → Create Target Group → Target Type: Instances → Protocol: HTTP → Port: 80 → Select VPC → Next → Register both EC2 instances → Create	A Target Group is a collection of EC2 instances that receive requests from the Load Balancer.
11	Create Application Load Balancer (ALB)	EC2 Dashboard → Load Balancers → Create Load Balancer → Application Load Balancer → Name → Internet-facing → Select VPC → Select both subnets → Create Security Group (Allow HTTP Port 80 from <code>0.0.0.0/0</code>) → Listener: HTTP:80 → Forward to Target Group → Create Load Balancer	The ALB distributes incoming HTTP requests evenly between the two EC2 instances.
12	Test the Load Balancer	After ALB is created → Copy DNS Name → Open browser: <code>http://<ALB-DNS-Name></code> → Refresh the page multiple times.	Each refresh alternates between the two EC2 instances, showing different Hostname and IP Address , confirming that the Load Balancer is working correctly.

PRGRAM 4

Step	Action	Procedure / Commands	Purpose / Expected Output
1	Login as Root User	Login using the AWS Root Account (not an IAM user).	Only the Root user can perform all IAM administration tasks.
2	Create IAM User	AWS Console → IAM → Users → Create User → Enter User Name (e.g., <code>test-user</code>) → Next → Add User to Group (optional) → Create User	Creates a new IAM user with no permissions initially.
3	Enable Console Access	Select the created user → Security Credentials → Enable Console Access → Select Custom Password → Apply	Allows the IAM user to log in to the AWS Console.
4	Save Login Details	Copy and save the Console Sign-in URL , Username , and Password .	Required for logging in as the IAM user.
5	Login as IAM User	Open the Console Sign-in URL → Enter Username & Password → Try opening S3.	You will receive Access Denied because no S3 permissions are assigned yet.
6	Create IAM Policy	Login again as Root User → IAM → Policies → Create Policy → Choose JSON → Paste the policy below → Next → Enter Policy Name (e.g., <code>S3AccessPolicy</code>) → Create Policy	Creates a custom policy that grants S3 permissions.
7	Attach Policy to User	IAM → Users → <code>test-user</code> → Add Permissions → Attach Policies Directly → Select <code>S3AccessPolicy</code> → Next → Add Permissions	Grants the IAM user permission to access S3.
8	Verify Permission	Login again as IAM User → Open S3 → Create a Bucket.	If successful, the policy is working correctly.
9	Remove Policy (Optional)	IAM → Users → <code>test-user</code> → Permissions → Select Attached Policy → Remove/Delete	Removes S3 access from the user.
10	Create IAM Group	IAM → User Groups → Create Group → Enter Group Name → Create Group	Creates a group to manage permissions for multiple users.
11	Add User to Group	Open the Group → Add Users → Select <code>test-user</code> → Add User	The user becomes a member of the IAM group.
12	Attach Policy to Group	Open Group → Permissions → Add Permissions → Attach Policies → Select <code>S3AccessPolicy</code> → Add Permissions	All users in the group inherit the S3 permissions.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3Access",
      "Effect": "Allow",
      "Action": [
        "s3:ListAllMyBuckets",
        "s3:ListBucket",
        "s3:GetObject",
        "s3:PutObject",
        "s3>DeleteObject",
        "s3:CreateBucket"
      ],
      "Resource": [
        "arn:aws:s3:::*",
        "arn:aws:s3:::*/*"
      ]
    }
  ]
}

```

PROGRAM 5

Step	Action	Procedure / Commands	Purpose / Expected Output
1	Install MySQL Workbench	Open Browser → Search MySQL Workbench → Download → Install according to your OS.	Installs the MySQL client used to connect to the RDS database.
2	Open RDS Service	AWS Console → Search RDS → Databases → Create Database	Starts the RDS database creation wizard.
3	Choose Database Engine	Select Standard Create → Engine: MySQL → Edition: MySQL Community → Version: 8.0.32 (or latest)	Creates a MySQL database in AWS.
4	Choose Template	Select Free Tier	Uses AWS Free Tier resources.
5	Configure Database	Enter DB Instance Identifier (e.g., <code>student-db</code>) → Enter Master Username (e.g., <code>admin</code>) → Enter Master Password	Creates the database with administrator credentials.

6	Storage & Connectivity	Keep Storage as default → Public Access = Yes → Availability Zone = No Preference → Click Create Database	Makes the database accessible from your local computer.
7	Get Database Details	Open the created database → Copy the Endpoint and Port (3306)	Required to connect using MySQL Workbench.
8	Connect Using MySQL Workbench	Open MySQL Workbench → Click + (New Connection) → Connection Name: RDS-MYSQL → Hostname: Paste Endpoint → Port: 3306 → Username: Master Username → Enter Password → Test Connection	Successfully connects to the AWS RDS database.
9	If Connection Fails	AWS Console → RDS Database → Connectivity & Security → Click VPC Security Group → Edit Inbound Rules → Add Rule → Type: MySQL/Aurora (3306) (or <i>All Traffic as per lab manual</i>) → Source: My IP (recommended) or Anywhere (0.0.0.0/0) (lab) → Save	Allows your computer to access the RDS instance.
10	Test Connection Again	Return to MySQL Workbench → Test Connection → OK	Connection should be successful.
11	Open SQL Editor	Open the saved connection in MySQL Workbench.	Ready to execute SQL queries.
12	Perform CRUD Operations	Execute SQL commands (below).	Verifies that the database is working correctly.

Connect Using MySQL CLI

Install MySQL Client (if not installed). Open **Command Prompt/Terminal** and run:

```
mysql -h <RDS-Endpoint> -P 3306 -u <Master-Username> -p
```

Example:

```
mysql -h student-db.abc123.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p
```

PROGRAM 6

Step	Action	Procedure / Commands	Purpose / Expected Output
1	Create RDS Database	AWS Console → RDS → Create Database → Standard Create → MySQL → MySQL Community → Free Tier → Enter DB Name, Master Username, Password → Public Access = Yes → (Optional) Specify Initial Database Name → Create Database	Creates the MySQL database in AWS RDS.
2	Copy RDS Endpoint	Open the RDS instance → Connectivity & Security → Copy the Endpoint and Port (3306) .	Required to connect from the EC2 instance.
3	Create VPC	VPC → Create VPC	Creates a virtual network.
4	Create Internet Gateway (IGW)	Internet Gateway → Create → Attach to VPC	Enables Internet connectivity.
5	Create Public Subnet	Create a subnet inside the VPC.	Hosts the EC2 instance.
6	Configure Route Table	Create Route Table → Associate with Subnet → Add Route 0.0.0.0/0 → Target = IGW	Makes the subnet public.
7	Launch EC2 Instance	EC2 → Launch Instance → Ubuntu → t2.micro → Create Key Pair → Select the created VPC & Subnet → Enable Auto Assign Public IP → Security Group: Allow SSH (22) from Anywhere → Launch.	Creates the web server.
8	Update Security Group	EC2 → Security Group → Edit Inbound Rules → Add:• SSH (22)• HTTP (80)• HTTPS (443)• MySQL (3306) (<i>if required by the lab</i>)	Allows SSH, web traffic, and database communication.
9	Connect to EC2 using SSH	Change key permission: <pre>chmod 400 mykey.pem\n\nConnect:bash\nssh -i mykey.pem ubuntu@<EC2-Public-IP>\n</pre>	Logs in to the EC2 instance.
10	Install Apache, PHP & MySQL Extension	<pre>sudo apt update -y\nsudo apt install -y apache2 php libapache2-mod-php php-mysql mysql-client\n\nsudo systemctl enable apache2\n\nsudo systemctl start apache2</pre>	Installs the web server, PHP, MySQL driver, and MySQL client.

11	Connect to RDS using MySQL CLI	<pre>mysql -h <RDS-Endpoint> -P 3306 -u <Master-Username> -p</pre> Example: <pre>mysql -h studentdb.abcd123.ap-south-1.rds.amazonaws.com -P 3306 -u admin -p</pre>	Verifies that EC2 can connect to the RDS database.
12	Create Database/Table (if needed)	<pre>CREATE DATABASE StudentDB;\nUSE StudentDB;\nCREATE TABLE Student(\n id INT\n PRIMARY KEY,\n name VARCHAR(50),\n dept VARCHAR(30)\n);\nINSERT INTO Student\nVALUES\n(1, 'Rahul', 'MCA'),\n(2, 'Anita', 'MCA');</pre>	Creates sample data in RDS.
13	Create PHP File	<pre>sudo nano /var/www/html/index.php</pre>	Creates the PHP application.
14	Write PHP Code	Paste the code below and save (Ctrl+O, Enter, Ctrl+X).	PHP connects to RDS and displays the records.
15	Open in Browser	Open: <code>text\http://<EC2-Public-IP>/index.php</code>	Displays the data stored in the RDS database.

```
<?php
```

```
$host = "YOUR_RDS_ENDPOINT";
```

```
$user = "admin";
```

```
$password = "YOUR_PASSWORD";
```

```
$dbname = "StudentDB";
```

```
$conn = new mysqli($host, $user, $password, $dbname);
```

```
if ($conn->connect_error) {
```

```
    die("Connection Failed: " . $conn->connect_error);
```

```
}
```

```
$sql = "SELECT * FROM Student";
```

```
$result = $conn->query($sql);
```

```
if ($result->num_rows > 0) {
```

```
    echo "<h2>Student Records</h2>";
```

```
    echo "<table border='1'>";
```

```
    echo "<tr><th>ID</th><th>Name</th><th>Department</th></tr>";
```

```
    while($row = $result->fetch_assoc()) {
```

```
        echo "<tr>";
```

```
        echo "<td>".$row["id"]."</td>";
```



```

        echo "<td>".$row["name"]."</td>";
        echo "<td>".$row["dept"]."</td>";
        echo "</tr>";
    }

    echo "</table>";
} else {
    echo "No Records Found";
}

$conn->close();
?>

```

PROGRAM 7

Step	AWS Service / Location	Action	Command / Code / Setting
1	VPC	Create a new VPC	AWS Console → VPC → Create VPC
2	Internet Gateway	Create IGW	VPC → Internet Gateways → Create
3	Internet Gateway	Attach IGW to VPC	Actions → Attach to VPC
4	Subnet	Create Public Subnet	VPC → Subnets → Create Subnet
5	Route Table	Create Route Table	VPC → Route Tables → Create Route Table
6	Route Table	Add Internet Route	Destination: 0.0.0.0/0 → Target: Internet Gateway
7	Route Table	Associate Subnet	Route Table → Subnet Associations → Select Public Subnet
8	EC2	Launch Instance	Select Ubuntu AMI, t2.micro, Key Pair
9	EC2 Network	Select Network	Choose VPC and Public Subnet
10	EC2 Network	Enable Public IP	Auto Assign Public IP = Enable
11	Security Group	Add SSH Rule	Type: SSH , Port: 22 , Source: Anywhere
12	Security Group	Add HTTP Rule	Type: HTTP , Port: 80 , Source: Anywhere
13	EC2	Launch Instance	Click Launch Instance
14	EC2	Copy Public IP	EC2 → Instance → Public IPv4 Address
15	Local Terminal	Change Key Permission	chmod 400 keypair.pem
16	Local Terminal	Connect to EC2	ssh -i keypair.pem ubuntu@<Public-IP>

Step	Purpose	Command
17	Update packages	<code>sudo apt update</code>
18	Install Apache	<code>sudo apt install apache2 -y</code>
19	Create folder	<code>mkdir temp</code>
20	Go inside folder	<code>cd temp</code>
21	Download Bootstrap website	<code>wget https://github.com/twbs/bootstrap/releases/download/v5.1.3/bootstrap-5.1.3-examples.zip</code>
22	Unzip files	<code>unzip bootstrap-5.1.3-examples.zip</code>
23	Enter extracted folder	<code>cd bootstrap-5.1.3-examples</code>
24	Copy website files	<code>sudo cp -r * /var/www/html/</code>
25	Enable Apache	<code>sudo systemctl enable apache2</code>
26	Start Apache	<code>sudo systemctl start apache2</code>
27	Check Apache Status	<code>sudo systemctl status apache2</code>

Step	Action
1	Open AWS S3
2	Create Bucket
3	Bucket type → General Purpose
4	Enter a globally unique bucket name
5	Create Bucket
6	Upload HTML, CSS, JS files
7	Open Properties
8	Enable Static Website Hosting
9	Set Index Document as <code>index.html</code>
10	Save Changes
11	Copy Website Endpoint URL

Step	Action
1	Bucket → Permissions
2	Disable/Allow Public Access (as required)
3	Open Bucket Policy
4	Use Policy Generator
5	Policy Type → S3 Bucket Policy
6	Effect → Allow
7	Principal → *
8	Action → s3:GetObject
9	ARN → arn:aws:s3:::bucket-name/*
10	Generate Policy
11	Copy & Paste into Bucket Policy
12	Save Changes

PROGRAM 8-----

Step	Action	Commands / Procedure	Explanation
1	Launch EC2 Instance	1. Login to AWS Console.2. Open EC2 service.3. Click Instances .4. Select your instance.5. Click Start Instance .6. Copy the Public IPv4 Address .Example: 13.235.xxx.xxx	This public IP is used by VS Code to connect to the EC2 server.
2	Verify Security Group	Select your EC2 Instance → Security tab → Click Security Group → Inbound Rules → Make sure SSH (Port 22) is allowed from My IP or Anywhere (0.0.0.0/0) for testing.	Without Port 22 open, SSH connection will fail.
3	Install Remote SSH Extension	Open VS Code → Click Extensions (Ctrl+Shift+X) → Search Remote - SSH → Click Install .	This extension allows VS Code to connect to remote Linux servers.
4	Move PEM Key	Copy your downloaded .pem file to a safe location.Example:Windows: C:\Users\YourName\.ssh\aws-key.pemLinux/macOS: ~/.ssh/aws-key.pem	The PEM file is your authentication key.
5	Set Permission (Linux/macOS/WSL only)	bash chmod 400 ~/.ssh/aws-key.pem	Gives read permission only to the owner. SSH requires this permission. Windows users can skip this step.
6	Open SSH Configuration File	Open VS Code.Press Ctrl + Shift + P .Type Remote-SSH: Open SSH Configuration File .Select the file:C:\Users\YourName\.ssh\conf	This file stores the SSH connection details.

		ig (Windows) or ~/.ssh/config (Linux/macOS).	
7	If .ssh Folder Does Not Exist	Open File Explorer. Go to C:\Users\YourName\. Create a new folder named .ssh . Inside it, create a file named config (without any extension).	VS Code will use this file to remember remote hosts.
8	Paste the Following Configuration	Replace the values with your own: text Host my-ec2 HostName 13.235.xxx.xxx User ubuntu IdentityFile C:\Users\YourName\.ssh\aws-key.pem	Host → Nickname. HostName → EC2 Public IP. User → ubuntu (Ubuntu AMI) or ec2-user (Amazon Linux). IdentityFile → Path to PEM file.
9	Save Config File	Press Ctrl + S .	Saves the SSH configuration.
10	Connect to EC2	Press Ctrl + Shift + P . Type Remote-SSH: Connect to Host . Select my-ec2 .	VS Code will start connecting.
11	Accept Fingerprint	If asked: Are you sure you want to continue connecting? Select Continue or type Yes .	This happens only the first time you connect.
12	Install VS Code Server	Wait for VS Code to automatically install the Remote Server on EC2.	This enables remote editing.
13	Verify Connection	Bottom-left corner of VS Code will display: SSH: my-ec2	Indicates successful connection.
14	Open Remote Folder	File → Open Folder → Select /home/ubuntu/ or another directory on EC2.	You can now edit files stored on the EC2 instance.
15	Open Terminal	Terminal → New Terminal	Opens a terminal directly on the EC2 instance.
16	Verify Connection	Execute: <code>bash pwd ls hostname</code>	Confirms you are working inside the EC2 machine.

Host my-ec2

HostName 13.235.xxx.xxx

User ubuntu

IdentityFile C:\Users\YourName\.ssh\aws-key.pem

PROGRAM 9

Step	Action	Command / Description
1	Create RDS	AWS → RDS → Create Database
2	Select Engine	MySQL
3	Template	Free Tier
4	Set Credentials	Username: admin, Password: your password
5	Public Access	Yes
6	Create Database	Click Create Database
7	Copy Endpoint	Example: db-1.c7o484agctqt.ap-south-2.rds.amazonaws.com
8	Create EC2	Ubuntu t2.micro/t3.micro
9	Connect EC2	ssh -i key.pem ubuntu@PUBLIC_IP
10	Update Packages	sudo apt update
11	Install Java	sudo apt install openjdk-17-jdk -y
12	Verify Java	java -version
13	Install MySQL Client	sudo apt install mysql-client -y
14	Connect to RDS	mysql -h db-1.c7o484agctqt.ap-south-2.rds.amazonaws.com -P 3306 -u admin -p
15	Create Database	CREATE DATABASE companydb;
16	Use Database	USE companydb;
17	Create Table	Create student table
18	Exit MySQL	exit
19	Create Java File	nano JdbcDemo.java
20	Paste Java Code	Save using Ctrl+X, Y, Enter
21	Download JDBC Driver	wget https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-9.4.0.tar.gz
22	Extract Driver	tar -xvzf mysql-connector-j-9.4.0.tar.gz
23	Find JAR	find . -name "*.jar"
24	Compile Program	javac -cp ./mysql-connector-j-9.4.0.jar JdbcDemo.java
25	Run Program	java -cp ./mysql-connector-j-9.4.0.jar JdbcDemo
26	Verify Output	Insert → Display → Update → Delete

```
import java.sql.*;
```

```

public class JdbcDemo {

    public static void main(String[] args) {

        String url = "jdbc:mysql://YOUR_RDS_ENDPOINT:3306/companydb";
        String user = "admin";
        String password = "yourpassword";

        try {

            Connection con = DriverManager.getConnection(url, user, password);

            // INSERT
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO student(name,age,sal) VALUES(?,?,?)");

            ps.setString(1, "Ram");
            ps.setString(2, "22");
            ps.setInt(3, 25000);

            ps.executeUpdate();
            System.out.println("Record Inserted");

            // DISPLAY
            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery("SELECT * FROM student");

            System.out.println("\nStudent Records");

            while (rs.next()) {
                System.out.println(
                    rs.getInt("id") + " " +
                    rs.getString("name") + " " +
                    rs.getString("age") + " " +
                    rs.getInt("sal"));
            }

            // UPDATE
            st.executeUpdate(
                "UPDATE student SET sal=30000 WHERE name='Ram'");

            System.out.println("\nRecord Updated");
        }
    }
}

```

```
// DELETE
st.executeUpdate(
    "DELETE FROM student WHERE name='Ram'");

System.out.println("Record Deleted");

con.close();

} catch (Exception e) {
    System.out.println(e);
}
}
```

Download JDBC Driver

wget <https://dev.mysql.com/get/Downloads/Connector-J/mysql-connector-j-9.4.0.tar.gz>

Extract driver

```
tar -xvzf mysql-connector-j-9.4.0.tar.gz
```

Verify

```
find . -name "*.jar"
```

Copile sours code

```
javac -cp ./mysql-connector-j-9.4.0/mysql-connector-j-9.4.0.jar javaJdbc.java
```

Run source code

```
java -cp ./mysql-connector-j-9.4.0/mysql-connector-j-9.4.0.jar javaJdbc
```

PROGRAM 10-----

Step	Service	What to Do
1	Security Group	Create one Security Group. Add SSH (22) from My IP and HTTP (80) from Anywhere
2	Target Group	Create Target Group → Type = Instances → HTTP → Port 80 → Default VPC
3	Target Group	When it asks to register instances, don't select anything → Create Target Group
4	Load Balancer	Create Application Load Balancer
5	Load Balancer	Internet Facing → IPv4
6	Load Balancer	Select Default VPC
7	Load Balancer	Select 2 default subnets
8	Load Balancer	Select the Security Group created in Step 1

9	Load Balancer	Listener HTTP:80 → Forward to Target Group
10	Launch Template	Create Launch Template
11	Launch Template	Amazon Linux 2
12	Launch Template	t2.micro
13	Launch Template	Select Key Pair
14	Launch Template	Select same Security Group from Step 1
15	Launch Template	Do NOT select any subnet
16	Launch Template	Paste User Data Script
17	Auto Scaling Group	Create Auto Scaling Group using Launch Template
18	Auto Scaling Group	Select Default VPC
19	Auto Scaling Group	Select the same 2 default subnets
20	Auto Scaling Group	Attach Existing Target Group
21	Auto Scaling Group	Enable ELB Health Checks
22	Auto Scaling Group	Min=1, Desired=2, Max=3
23	Auto Scaling Group	Create
24	EC2 Instances	Wait 2–3 minutes → 2 instances will be created automatically
25	Testing	Open Load Balancer DNS in browser
26	Demo	Terminate one instance
27	Demo	Wait 2–5 minutes → New instance automatically created

PROGRAM 1

Step	AWS Service	What to Do
1	VPC	Create VPC (CIDR: 10.1.0.0/16)
2	Internet Gateway	Create IGW and attach it to the VPC
3	Subnet	Create Public Subnet (10.1.1.0/24)
4	Subnet	Create Private Subnet (10.1.2.0/24)
5	Route Table	Create Route Table
6	Route Table	Add Route: Destination 0.0.0.0/0 → Target = Internet Gateway

7	Route Table	Associate the Route Table with the Public Subnet
8	Security Group	Create Security Group
9	Security Group	Add SSH (22) from Anywhere (or My IP)
10	EC2	Launch Instance
11	EC2	Select Amazon Linux 2 AMI
12	EC2	Select <code>t3.micro</code> (if <code>t2.micro</code> isn't available)
13	EC2	Select/Create Key Pair
14	EC2	Choose your VPC and Public Subnet
15	EC2	Enable Auto-assign Public IP
16	EC2	Select the Security Group
17	EC2	Launch Instance
18	EC2	Copy the Public IP
19	Terminal	<code>chmod 400 your-key.pem</code>
20	Terminal	<code>ssh -i your-key.pem ec2-user@<Public-IP></code>